09_ae_b

Data Quality

Q26. What is the difference between a test and a monitor in an analytics pipeline, and when do you reach for each?

Ideal answer. A test is a deterministic, blocking assertion that runs at build/deploy time against a known invariant - it either passes or fails the build (e.g. a `unique` test on a primary key). A monitor is a continuous, often statistical observation over production data that *alerts* rather than blocks, tolerating gradual drift you cannot encode as a hard pass/fail (e.g. "row volume is 4 sigma below the trailing 28-day mean"). The rule I use: if I can state the invariant precisely and a violation should stop the pipeline, it is a test; if the signal is anomalous-but-not-illegal and a human should triage it, it is a monitor. The failure of treating everything as a test is brittle pipelines that page on benign variance; the failure of treating everything as a monitor is silent corruption that ships because nothing blocked it. Mature stacks use both - tests guard contracts, monitors guard behavior. **Why Helios demonstrates it.** Helios encodes hard invariants as dbt tests (`revenue_reconciles` to the cent, funnel monotonicity, `accepted_values` on the 10 channel groups) and treats anomaly detection as a *monitor* - `stats-mcp.detect_anomaly` flags deviations for the Diagnose agent, it does not fail the build. **Follow-ups.** Where does dbt source `freshness` sit on the test-vs-monitor line, and why? / How would you avoid alert fatigue if every metric had a monitor? / Can a monitor's threshold itself be regression-tested?

Q27. Explain reconciliation as a data-quality control. Why is it stronger than schema tests alone?

Ideal answer. Reconciliation asserts that an aggregate computed at a downstream layer equals the same aggregate computed independently from the authoritative source, within a stated tolerance - it is an end-to-end conservation check across the whole transformation chain. Schema tests (`not_null` , `unique` , `accepted_values`) prove a column is well-typed and structurally valid, but a model can pass every schema test while silently dropping 12% of revenue because a join fanned out or a filter was too aggressive. Reconciliation catches exactly that class of *quantitative* drift that structural tests are blind to. The tradeoff is cost: reconciliation re-scans the source, so I scope it to the metrics that carry business and label weight (revenue, session counts) rather than everything. I treat the source total as ground truth and tolerate only a tiny, explicitly-justified delta. **Why Helios demonstrates it.** The `revenue_reconciles` generic test asserts `SUM(revenue)` in `fct_orders` equals `SUM(purchase_revenue_in_usd)` over raw `purchase` events with tolerance 0; at runtime `warehouse-mcp.reconcile` re-checks marts against canonical

totals and any drift >0.5% fails the finding before it reaches the Decision Brief. **Follow-ups.** Why tolerance 0 for the build test but 0.5% at runtime? / How do you reconcile a *rate* rather than a sum? / What do you do when reconciliation fails - block, quarantine, or degrade?

Q28. Funnel monotonicity: what is the invariant and how do you test it?

Ideal answer. A session that reached `add_to_cart` necessarily reached `view_item`, so the funnel must satisfy `sessions ≥ reached_view_item ≥ reached_add_to_cart ≥ … ≥ reached_purchase` - counts are non-increasing down the funnel and every step rate is therefore <= 1. I enforce this by computing the `reached_*` flags as *max-downstream* (a flag is true if the session reached that step or any later step), which makes monotonicity structural rather than hoped-for, then I add a singular test that fails if any adjacent pair violates the ordering:

```sql
-- tests/assert_funnel_monotonic.sql  (fails ⇒ returns rows)
select event_date
from {{ ref('fct_daily_funnel') }}
where view_item_sessions      > sessions
   or add_to_cart_sessions    > view_item_sessions
   or begin_checkout_sessions > add_to_cart_sessions
   or purchasing_sessions     > add_payment_info_sessions
```

Without this, a sessionization bug or a non-monotonic flag produces step rates above 1, which silently poisons the mix/rate decomposition. **Why Helios demonstrates it.** Helios defines funnel flags as `reached_*` max-downstream in `int_ga4__funnel_steps` (the retired `did_*` naming was non-monotonic) precisely so the decomposition's segment rates `r_i` are always valid probabilities; the monotonicity test is a keystone because it fails silently otherwise. **Follow-ups.** Why is making the property structural better than only testing for it? / How would a late-arriving `purchase` event break monotonicity, and does your incremental strategy handle it? / Could you express this as a dbt relationship/expression test instead of a singular test?

Q29. How do you implement and reason about data freshness SLAs?

Ideal answer. Freshness is a contract on *latency*: the newest data must reach the consumption layer within a bounded window, with a warn threshold and a hard error threshold. I declare it on the source with a `loaded_at_field` and `warn_after` / `error_after` so dbt computes the gap between the latest loaded record and now, and I make a freshness *error* block the downstream consumer rather than letting it analyze stale data. The reasoning is that a diagnosis run on yesterday's incomplete shard is worse than no diagnosis - it produces a confident, wrong "why". The tradeoff is choosing thresholds tight enough to catch a broken pipeline but loose enough to absorb normal late-landing; for GA4 I use 36h warn / 48h error because GA4 events land late. **Why Helios demonstrates it.** Helios declares source freshness on `src_ga4` (`warn_after: 36h`, `error_after: 48h`); a freshness error blocks the agent pipeline so the autonomous run never

diagnoses a stale or partial day - protecting the "<5 min, correct" promise rather than just speed. **Follow-ups.** The public GA4 dataset is static - so why bother with freshness at all? / How does freshness interact with your 3-day incremental lookback? / What's the difference between freshness and completeness, and which is harder to guarantee?

Q30. What is the "silent failure" problem in data pipelines, and how do you design against it?

Ideal answer. Silent failure is when a transformation produces output that is structurally valid and looks plausible but is quantitatively wrong - no error is raised, the build is green, and a human acts on a corrupted number for weeks. It is the most dangerous failure mode in analytics because the cost is unbounded and the detection is accidental. I design against it three ways: make invariants *structural* where possible (so the bad state cannot be represented), add *conservation* tests (reconciliation) that catch quantitative drift schema tests miss, and put the highest-leverage transforms under *golden tests* with hand-worked expected outputs. The mental model: the more downstream consumers trust a number blindly, the louder its upstream transform must fail when wrong. **Why Helios demonstrates it.** Helios explicitly tags five keystones that "fail silently" - sessionization, the semantic registry, `decompose_change`, revenue reconciliation, and `fct_daily_funnel` carrying revenue - and gives each a golden or reconciliation test plus the CI eval gate, so a silent corruption surfaces as a red build, not a wrong brief. **Follow-ups.** Give an example of a bug that passes every schema test but is still wrong. / How does the offline eval benchmark act as a silent-failure detector for the agents? / Why are keystone transforms more likely to fail silently than leaf models?

Q31. When and how do you use `accepted_values` tests, and what are their limits?

Ideal answer. `accepted_values` asserts a column's domain is exactly a known, closed set - it is the right test for categorical dimensions whose cardinality is fixed by a business definition, because it catches both new unexpected values (a logic gap) and a typo'd mapping. I use it where the set is *canonical and closed*, not merely *currently observed*; the anti-pattern is pinning a high-cardinality or growing field, which turns the test into a maintenance treadmill that fails on every legitimate new value. Its limit is that it validates membership, not distribution - a column can be 100% one accepted value when it should be spread across ten, and `accepted_values` passes happily. So I pair it with a distribution monitor when the spread itself matters.

```yaml
- name: channel_group
  tests:
    - accepted_values:
        values: ['Direct','Organic Search','Paid Search','Display','Paid Social',
                 'Organic Social','Email','Affiliates','Referral','Other']
```

Why Helios demonstrates it. Helios pins `dim_channels.channel_group` to exactly the 10 GA4 default channel groups via `accepted_values` - this is the test that fails loudly if the `channel_group_case()` macro ever emits an 11th group or a "Paid Other" leak, keeping the single source of truth honest. **Follow-ups.** Why is `accepted_values` on `channel_group` better placed on the dim than on every fact? / How would you catch the "everything collapsed to one value" failure that `accepted_values` misses? / Would you ever generate the accepted set from a seed instead of hardcoding it?

Q32. How do you decide what tolerance to set on a reconciliation or range test?

Ideal answer. Tolerance should be derived from a known, explainable source of legitimate difference, never tuned to "make the test pass". I start at zero and only widen it for a documented reason: floating-point summation order, a deliberate rounding boundary, or a known late-arrival window. If I cannot name why a delta exists, the delta is a bug, not a tolerance. The danger of a loose tolerance is that it becomes a hiding place - a 5% band quietly absorbs a real 4% revenue leak. So I keep build-time exactness strict (the same data through two paths should match exactly) and reserve looser runtime tolerances for genuinely noisy cross-system comparisons, with the tolerance value itself reviewed in code review. **Why Helios demonstrates it.** Helios uses tolerance 0 on the build-time `revenue_reconciles` test (same warehouse, two query paths must match to the cent) but a 0.5% drift ceiling at runtime via `warehouse-mcp.reconcile`, because runtime compares governed marts against canonical totals where rounding and edge filters can introduce a tiny, bounded difference. **Follow-ups.** Walk me through justifying a non-zero tolerance to a skeptical reviewer. / How do absolute vs relative tolerances behave differently at low volumes? / Could you make tolerance scale with sample size?

Q33. How do you test data quality on a column that is mostly null, like GA4's `user_id`?

Ideal answer. First, I stop treating high-nullity as a defect to be tested away and instead model around it honestly - I document the expected null rate and pick a different grain. Testing `user_id is not null` would fail 99% of rows and tell me nothing useful. Instead I add a *bounded* data test: assert the null rate stays within an expected band (so a sudden drop to 0% non-null is caught as an instrumentation break), and route all identity logic through the field that *is* populated. The senior move is to make the limitation explicit in the model contract and metric definitions so consumers know the metric is an approximation, not to paper over it with a green test. **Why Helios demonstrates it.** In Helios `user_id` is almost always null, so identity is cookie-grain on `user_pseudo_id` and user-level metrics (`users`, ARPU) are documented as cookie-based approximations - the data model treats nullity as a stated assumption, not a test failure to suppress. **Follow-ups.** How would you alert on `user_id` *coverage* changing rather than its value? / What's the downstream metric distortion from cookie-grain identity, and how do you bound it? / Would you use a `not_null` test with a configured threshold (e.g. dbt_utils)?

Q34. How do you prevent data-quality issues in the source from being misdiagnosed as a business anomaly?

Ideal answer. A pipeline that cannot distinguish "the funnel moved" from "the data about the funnel changed" will confidently explain artifacts, which is worse than missing real signal. I build a data-quality gate *upstream* of diagnosis - freshness, completeness, and reconciliation must pass before any anomaly is interpreted - and I add a data-quality hypothesis to the root-cause search itself, so a tracking outage, a deployment-shaped step in a single channel, or a schema change is considered as a candidate explanation and can be ruled out. The principle is verify-then-trust: prove the data is sound before pricing a movement in dollars. **Why Helios demonstrates it.** Helios's Critic agent runs an explicit *data-quality* refutation (alongside mix-confound, insufficient-sample, and seasonality) and can DOWNGRADE or DROP a finding it judges an artifact; the eval benchmark even includes a dedicated data-quality-artifact bucket so the pipeline is graded on correctly *declining* to diagnose noise. **Follow-ups.** What does a tracking-outage anomaly look like in a decomposition, and how is it distinguishable from a real drop? / How do you keep the data-quality check from suppressing real anomalies? / Where does this gate live relative to the dry-run cost gate?

Testing

Q35. Differentiate schema tests, data tests, unit tests, and golden tests. When is each the right tool?

Ideal answer. Schema tests assert structural contracts on columns (`unique`, `not_null`, `relationships`, `accepted_values`) - cheap, run on every build, catch type/key violations. Data tests assert *values are sane* over real data (ranges, freshness, row-count thresholds) - they catch plausible-but-wrong content schema tests pass through. Unit tests assert that a transform maps a *seeded synthetic input* to an *asserted output* in isolation, independent of production data - they pin business logic like sessionization. Golden tests are a unit test whose expected output is a hand-worked, authoritative reference value for a high-stakes computation. The rule: schema and data tests guard the data flowing through; unit and golden tests guard the *logic* itself, which is the only way to test a transform before any data exists. **Why Helios demonstrates it.** Helios's testing pyramid uses all four - `accepted_values` / `relationships` (schema), revenue-range and conversion-rate-bounds (data), seeded `given` / `expect` unit tests on the sessionizer and channel logic, and golden tests with hand-worked mix/rate/interaction values for `stats-mcp.decompose_change`. **Follow-ups.** Why can't a unit test replace a reconciliation test? / Which of these can run with zero warehouse cost, and why does that matter for CI? / Where do golden tests fit relative to property-based tests?

Q36. How do golden tests work for the decomposition, and why are they essential there specifically?

Ideal answer. A golden test fixes a small, fully-specified input and an expected output that I computed by hand (or analytically) and verified independently, then asserts the implementation reproduces it exactly within numerical tolerance. They are essential for the decomposition because mix/rate/interaction is the analytical centerpiece - a sign error or a wrong base period (`t0` vs `t1` weights) is *plausible* and would silently flip a diagnosis from "real behavior change" to "composition artifact". I construct goldens that exercise the adversarial cases, not just the happy path: a pure mix shift (segment rates held constant, only weights move), a pure rate change, and a Simpson's-paradox case where the aggregate falls while every segment rises. If the implementation passes all three with the additive identity `dR = mix + rate + interaction` holding, the math is trustworthy.

```
# pure mix shift: rates fixed, weights move ⇒ rate_effect must be ~0
# Simpson case: aggregate down, every segment up ⇒ mix_effect dominates negative
assert abs((mix + rate + interaction) - delta_R) < 1e-9
assert abs(rate_effect) < 1e-9          # pure-mix scenario
```

Why Helios demonstrates it. `stats-mcp.decompose_change` is a named keystone unit-tested against hand-worked golden values, and the eval injector exploits the same separation (rate-perturbation changes only `r_i`, volume-perturbation changes only `w_i`) so the golden tests and the benchmark validate the same identity from two directions. **Follow-ups.** Why use `t0` weights for the rate effect rather than an average of `t0` / `t1`? / How do you pick the numerical tolerance for a float golden? / What does a failing Simpson's-paradox golden tell you about the bug?

Q37. What does TDD look like applied to SQL transformations?

Ideal answer. TDD on SQL means writing the assertion before the model: I declare the grain and the contract, write the failing test (a unit test with seeded rows and expected output, or a schema/relationship test), then write the SQL until the test goes green. It works especially well for transforms with a crisp definition - sessionization, funnel flags, channel grouping - because the test *is* the spec and forces me to pin edge cases (a session spanning midnight, a null medium) up front rather than discovering them in production. The tradeoff is that not all SQL is amenable to cheap unit testing - large aggregations are better guarded by reconciliation - so I apply TDD where the logic is subtle and the input can be small and synthetic. The discipline pays off most on the transforms that fail silently. **Why Helios demonstrates it.** Helios's operating rule is "TDD on SQL: write the dbt/golden test first, then make it pass," applied specifically to the keystone transforms (sessionization, `reached_*` flags) whose correctness every downstream number depends on. **Follow-ups.** What's a sessionization edge case you'd write a failing test for first? / How do dbt `unit_tests` (`given` / `expect`) let you TDD without touching the warehouse? / When is TDD on SQL not worth it?

Q38. Explain the "keystone test budget" - how do you decide where to invest scarce testing effort?

Ideal answer. Tests are not free - they cost authoring time, build time, and warehouse bytes - so I spend the budget where the product of *blast radius* and *silent-failure probability* is highest, not uniformly. A keystone is a transform that (a) feeds many downstream consumers and (b) can be wrong while looking right; those get the heavy artillery - golden tests, reconciliation, unit tests. Leaf models that are visibly broken when wrong, or that one report consumes, get light schema tests. This is risk-weighted testing: I'd rather have five bulletproof tests on the sessionizer and the decomposition than two hundred shallow `not_null` tests spread evenly. The anti-pattern is a high test *count* that gives false confidence while the load-bearing logic is untested. **Why Helios demonstrates it.** Helios names exactly five keystones (sessionization, the semantic registry, `decompose_change`, revenue reconciliation, `fct_daily_funnel` carrying revenue) and concentrates golden/reconciliation tests there - "get these right, test them hard, they fail silently." **Follow-ups.** How do you identify a keystone from the DAG topology? / Doesn't concentrating tests leave leaf models under-covered - is that acceptable? / How would you measure whether your test budget is well-allocated?

Q39. How do you gate CI on analytics changes without making the pipeline slow or expensive?

Ideal answer. I make CI fast and cheap by building only what changed and its descendants, deferring everything else to a production manifest, and capping bytes. In dbt that's slim CI: `state:modified+` selects modified models plus children, `--defer --state` resolves unchanged refs against the prod manifest so I don't rebuild the world, and tests run on the same selection. I add a byte guard so a runaway query is killed by the warehouse, not discovered on the bill. The principle is that CI must be cheap enough to run on *every* PR or it gets bypassed - a gate people route around is not a gate. The tradeoff is manifest staleness, which I manage by refreshing the prod manifest on merge.

```
- run: dbt build --select state:modified+ --defer --state ./prod-manifest
- run: dbt test  --select state:modified+
- run: python eval/run_benchmark.py    # accuracy gate
```

Why Helios demonstrates it. Helios runs slim CI (`state:modified+ --defer`) plus the eval harness on every PR, and bounds cost by dry-running every benchmark scenario first and aborting if total scanned bytes exceed the per-run budget - a 12-scenario smoke subset on every push, the full set on PRs to `main` . **Follow-ups.** What goes stale in the deferred prod manifest and how do you keep it fresh? / Why run a smoke subset on push but the full suite only on PR? / How do you keep CI deterministic when the agents involve an LLM?

Q40. In what sense is the eval harness an integration test, and how does it differ from your unit tests?

Ideal answer. Unit tests prove a component is correct in isolation; an integration test proves the assembled system produces the right end-to-end outcome through all its seams. The eval harness is exactly that - it exercises the full path (semantic SQL -> warehouse -> stats -> agents -> finding) against inputs where I know the right answer, and grades the *system-level* output, not any one function. It catches failures unit tests cannot: a correct `decompose_change` wired to the wrong dimension, a registry change that subtly shifts a metric, an agent that drills the mix effect instead of the rate effect. The difference is the assertion target - a unit test asserts `f(x)=y`; the eval asserts "given a hidden anomaly, the system rediscovers the segment I injected". **Why Helios demonstrates it.** Helios's offline labeled benchmark injects known anomalies into a frozen GA4 copy and grades whether the pipeline rediscovers the root-cause segment (target >=85% vs <=45% naive); that is an integration test of the whole diagnosis chain, distinct from the golden unit test on `decompose_change` alone. **Follow-ups.** What integration bug would the eval catch that all your unit tests would miss? / How do you keep an integration test that involves an LLM stable enough to gate on? / Why inject anomalies rather than test against historical labeled incidents?

Q41. How do you test non-deterministic, LLM-in-the-loop components so CI remains a reliable gate?

Ideal answer. I push determinism to the boundaries and grade on outcomes, not on exact token output. Concretely: all math and SQL are deterministic tools (seeded), so the only stochastic element is the LLM's *choice* of which tool calls to compose - and I grade whether the final finding is correct (right segment, right mix-vs-rate verdict), which is robust to wording variation. I run a labeled benchmark with a tolerance band rather than asserting a single byte-identical answer, set temperature low for the graded run, and gate on aggregate accuracy with a regression tolerance instead of demanding 100%. The mental model: make the *verifiable* parts deterministic and the *judgment* parts measurable, so non-determinism never reaches the assertion. **Why Helios demonstrates it.** In Helios the LLM never authors SQL or computes a statistic (those are deterministic via `semantic-mcp` / `stats-mcp`), so CI gates on benchmark accuracy with a 2-point regression tolerance against a committed `main` baseline rather than on exact agent text - non-determinism is contained, not gated on. **Follow-ups.** Why a 2-point regression band rather than a hard floor? / How do you debug a benchmark regression when the cause is the LLM's tool-call choice? / Could a flaky single scenario make CI unreliable, and how do you handle it?

Q42. What is the right regression-gating policy for an accuracy metric in CI?

Ideal answer. A regression gate should fail a change that makes the system meaningfully worse than the last known-good state, comparing against a *committed baseline* rather than an absolute

number alone, because absolute targets drift and noise causes false fails. I gate on two things: an absolute floor (don't ship below the minimum bar) and a relative tolerance (don't regress more than N points versus the committed baseline), and I store the baseline in the repo so the comparison is auditable and updated deliberately on merge. The tolerance absorbs benign run-to-run noise; setting it too tight produces flaky red builds, too loose lets slow erosion through. Critically, I also gate the *non-negotiables* at zero tolerance - a hallucinated column is never an acceptable regression. **Why Helios demonstrates it.** Helios stores thresholds in `eval/gates.yaml` with `regression_tolerance: 0.02` against `eval/baselines/main.json`, failing the PR if top-1 drops >2 points or if any hallucination appears - a relative gate for accuracy, a zero-tolerance gate for governance. **Follow-ups.** When and how do you update the committed baseline? / Why is hallucination gated at zero but accuracy at a tolerance? / How would you prevent a series of within-tolerance regressions from silently eroding accuracy over many PRs?

Lineage

Q43. What is a data lineage DAG and what concrete problems does it let you solve?

Ideal answer. A lineage DAG is the directed acyclic graph of dependencies among data assets - sources, models, and consumers - where edges encode "is built from". It solves several concrete problems: build ordering (a topological sort tells you what to run before what), impact analysis (what breaks if I change X), root-causing a bad number (trace upstream to the first wrong model), and onboarding (the DAG is the map of the system). dbt derives this graph automatically from `ref()` and `source()` calls, so lineage is a byproduct of writing models correctly rather than a separate diagram that rots. The leverage is that a question like "what feeds this metric?" becomes a graph query, not an archaeology project. **Why Helios demonstrates it.** Helios's strict five-layer DAG (`src_ga4` -> staging -> intermediate -> marts -> semantic) gives model- and column-level lineage; the build order in the dependency map is a valid topological sort of exactly this graph, which is why no artifact is built before its dependencies. **Follow-ups.** Column-level vs model-level lineage - when do you need the finer grain? / How does `ref()` / `source()` make lineage a byproduct instead of separate documentation? / How would you detect an accidental cycle?

Q44. What are dbt exposures and why are they valuable?

Ideal answer. An exposure is a declared node representing a *downstream consumer* of the data - a dashboard, an application, an ML job, a report - that you register in the dbt project so it becomes part of the lineage graph even though dbt doesn't build it. Their value is that they close the loop on impact analysis: without exposures, lineage ends at the last model and you can't answer "if I change this mart, which business-facing surfaces are affected?". With them, you can select the full subgraph feeding a consumer, run only what a given deliverable needs, and communicate ownership and SLA. They turn the DAG from an internal build graph into a contract

with the things people actually look at. **Why Helios demonstrates it.** Helios declares its seven agents and the Decision Brief as exposures in `exposures.yml`, so `dbt ls --select +exposure:helios_decision_brief` answers "what feeds the brief?" and a change can be impact-analyzed all the way to the consuming agent before it ships. **Follow-ups.** What metadata should an exposure carry to be useful (owner, maturity, URL)? / How do exposures change your CI selection logic? / Can an exposure depend on the semantic layer rather than a raw mart, and why prefer that?

Q45. How do you perform impact analysis before shipping a model change?

Ideal answer. Impact analysis answers "what does this change touch?" before it touches it. I use the lineage graph to select the changed model plus its full downstream closure, including exposures, and build/test exactly that subgraph - so I see every affected mart, metric, and consumer rather than guessing. The pattern is `+changed_model+` to get upstream context and all descendants, intersected with `exposure:*` to surface business-facing impact. This converts a risky "I think this only affects X" into a verified blast radius, and it lets reviewers reason about scope. The discipline matters most for shared upstream models like sessionization, where a one-line change can ripple to every fact and every agent.

```
dbt ls    --select +int_ga4__sessionized+      # full blast radius
dbt build --select int_ga4__sessionized+ exposure:*   # build descendants + consumers
```

Why Helios demonstrates it. Helios uses `dbt build --select +<changed_model>+exposure:*` so a change to a keystone like `int_ga4__sessionized` shows everything it touches - every mart and every agent exposure - before merge, backed by the CI eval gate as the final blast-radius check. **Follow-ups.** Why include exposures in the selection rather than just models? / How does impact analysis interact with slim CI's `state:modified+`? / What change would have the largest blast radius in your DAG, and how would you de-risk it?

Q46. How does lineage support data governance and access control?

Ideal answer. Governance is about controlling *what* logic is authoritative and *who* can reach which data, and lineage is the backbone of both. Lineage lets you prove provenance (this number derives from these governed models, not an ad-hoc query), enforce that sensitive sources only flow through approved transforms, and scope credentials by layer so a consumer physically cannot bypass governance to hit raw data. The strongest control is structural: grant the consuming service read access only to the governed layer, so even a compromised or misbehaving consumer cannot reach raw facts. Lineage makes that boundary auditable - you can see, and test, that nothing reaches a consumer except through the governed path. **Why Helios demonstrates it.** Helios backs `semantic-mcp` with a read-only service account scoped to the

`helios_semantic` dataset only, so the LLM *physically cannot* query raw or mart tables - the governance boundary is enforced by IAM along the lineage, not just by policy. **Follow-ups.** Why enforce the boundary with IAM rather than just instructing the agent not to query raw data? / How would you audit that no consumer bypasses the governed layer? / Where do column-level access controls (PII) fit into this?

Q47. How do you version and evolve data models and metric definitions without breaking consumers?

Ideal answer. I treat models and metric definitions as a versioned contract: changes go through PRs with impact analysis, breaking changes are flagged explicitly, and consumers reference *named, governed* metrics rather than raw SQL so a definition can evolve in one place. For genuinely breaking changes I use additive evolution where possible (introduce the new definition alongside the old, migrate consumers, then retire) and dbt's model contracts/versions to make a schema a promise CI enforces. The principle is that the cost of a change should be borne at author time through review and tests, not at consume time through a silent break. Version control plus the DAG means every definition has history and every change has a reviewable blast radius. **Why Helios demonstrates it.** Helios centralizes every metric in `semantic_models.yml` and channel logic in one macro, so a definition changes in exactly one governed place and CI's eval gate catches any consumer-visible regression; the retirement of the non-monotonic `did_*` flags in favor of `reached_*` is a documented, repo-tracked breaking-change migration. **Follow-ups.** How do dbt model contracts turn a schema into an enforced promise? / Walk through deprecating a metric with live consumers. / How does the eval baseline detect an unintended semantic drift from a "safe" definition change?

Q48. Explain "the registry as a contract." What does that buy you?

Ideal answer. A metric/dimension registry-as-contract means there is exactly one machine-readable definition of every metric and dimension, referentially compiled, that all SQL must be composed from - so the registry is not documentation, it is the enforced interface between intent and execution. It buys three things: a single source of truth (a metric is defined once, so two reports cannot disagree on what "conversion rate" means), structural elimination of invented columns (only registered names compile; an unknown name is a hard error, not free-text SQL), and safe evolution (change the contract, and every consumer follows). The tradeoff is upfront rigor - you must register a metric before you can query it - which is exactly the friction that prevents the sprawl of ad-hoc, subtly-divergent definitions. **Why Helios demonstrates it.** Helios's `semantic_models.yml` (28 metrics + 16 dimensions, referential-integrity compiled, 1:1 with dbt MetricFlow) is the sole SQL author via `semantic-mcp.build_query`; an unrecognized name is a hard error, which is how Helios achieves 0 hallucinated columns structurally rather than by hoping the LLM behaves. **Follow-ups.** How is registry-as-contract different from just having a metrics

layer? / Why does "unknown name is a hard error" matter more than a warning? / How does the registry map to dbt MetricFlow, and why align them 1:1?

Q49. How is the CI eval gate a "regression firewall," and what would happen without it?

Ideal answer. A regression firewall is an automated barrier that prevents a known-good capability from silently degrading - it makes correctness a *merge condition*, not an aspiration. The eval gate runs the labeled benchmark on every PR and blocks the merge if accuracy regresses beyond tolerance or any governance invariant breaks, so a refactor that "looks fine" but quietly drops root-cause accuracy can never reach `main`. Without it, accuracy erodes the way untested behavior always does: each change is individually plausible, no one re-runs the full benchmark by hand, and three months later the system is meaningfully worse with no single commit to blame. The firewall converts that slow, invisible decay into a loud, attributable red build on the exact PR that caused it. **Why Helios demonstrates it.** Helios's CI eval gate (`.github/workflows/ci.yml` + `eval/gates.yaml`) is explicitly the regression firewall: once green, no change may drop top-1 accuracy >2 points or introduce any hallucination - the 85%-vs-45% result is defended automatically on every commit, not just claimed once. **Follow-ups.** Why is "every PR" essential - what breaks if it runs only nightly? / How do you keep the firewall itself trustworthy (i.e. test the tests)? / What's the failure mode if the baseline is updated carelessly?

Q50. Tie it together: how do data quality, testing, and lineage combine into a single trust story for an autonomous system?

Ideal answer. For an autonomous system that acts without a human in the loop, trust cannot be asserted - it has to be engineered into the pipeline as enforceable structure. The three layers compose: lineage defines and constrains the *paths* data can take (governed layer only, auditable provenance, impact-analyzable change); testing proves the *logic* on those paths is correct (golden tests on the centerpiece math, reconciliation against source, monotonicity invariants); and data-quality controls guard the *inputs and outputs* at runtime (freshness, reconciliation drift, the data-quality refutation). Each layer catches a different failure class, and the CI eval gate ties them together by proving the assembled system still produces the right end-to-end answer. The thesis is that the hard part of an AI analyst is not generating insight - it is making insight *correct and trusted*, and that is an analytics-engineering problem before it is a modeling one. **Why Helios demonstrates it.** Helios's three defensibility pillars are exactly this stack - governed-SQL grounding (lineage + registry), deterministic statistics (golden-tested tools), and adversarial verification plus the offline eval (the Critic + the regression-firewall benchmark) - which is why Helios is positioned as an engineered trust system, not a clever prompt. **Follow-ups.** If you could keep only one of the three layers, which and why? / How would this trust stack change if Helios ran on live, multi-tenant data? / Where is the weakest link in this story today, and how would you harden it?